

Семинар 1 · Разворачиваем стек и знакомимся с данными

Базы знаний и базы данных · кафедра ИУ-1 · 3 курс · неделя 1 · 90 минут

Цель	К концу семинара у вас работает PostgreSQL в Docker, загружен учебный датасет, написаны первые запросы, настроен репозиторий и собрана команда для проекта.
Чек-поинт 1	Ваш репозиторий (GitHub / GitLab / архив) с работающим docker compose up и файлом sql/seminar01.sql с решениями задач 1-7. Показывается преподавателю на семинаре или ссылка присылается в чат до семинара 2.
Баллы	2 балла (из 30 за чек-поинты). В срок — 2, позже — 1.
Что нужно	Ноутбук, Docker Desktop (или Docker Engine + Compose), Git, любой SQL-клиент: psql, DBeaver, DataGrip или встроенный в VS Code.

Как устроен семинар

Пять частей по 15-25 минут. Части 1-3 обязательны для чек-поинта. Часть 4 — организационная. Часть 5 — задачи со звёздочкой для тех, кто закончил раньше: они не влияют на балл, но пригодятся на неделе 3.

Автопроверки на первом семинаре нет: решения смотрим вместе, руками. Всё, что вы пишете сегодня, — начало вашего проекта. Схема из этого семинара — заготовка, которую вы перепроектируете под свою предметную область к неделе 4.

Часть 1 · Поднимаем стек (15 мин)

1.1. Получите шаблон репозитория kbdb-seminar01 (ссылка на GitHub или архив — в чате группы). Если это репозиторий — нажмите **Use this template** или сделайте fork и склонируйте себе; если архив — распакуйте и выполните `git init`. Дальше работаем в своей копии:

```
git clone <ссылка на вашу копию>      # или: unzip kbdb-seminar01.zip && cd kbdb-seminar01 && git init
cd kbdb-seminar01
```

Аккаунт на GitHub/GitLab желателен, но не обязателен на первой неделе: сдать можно и архивом. К проекту (неделя 4) репозиторий понадобится точно.

1.2. В корне репозитория уже лежит `docker-compose.yml`. Изучите его — сегодня нам нужен только один сервис, остальные будут добавляться по ходу курса:

```
services:
  postgres:
    image: timescale/timescaledb:latest-pg17
    container_name: kbdb-postgres
    environment:
      POSTGRES_USER: student
      POSTGRES_PASSWORD: student
      POSTGRES_DB: plant
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./sql/init:/docker-entrypoint-initdb.d # схема и датасет загрузятся сами

volumes:
  pgdata:
```

1.3. Запустите стек и убедитесь, что база отвечает:

```
docker compose up -d
docker compose ps          # STATUS должен быть running / healthy
docker compose exec postgres psql -U student -d plant -c "select version();"
```

1.4. Подключитесь любым SQL-клиентом: host localhost, port 5432, db plant, user/password student. Если предпочитаете консоль — `docker compose exec postgres psql -U student -d plant`.

Если что-то не так

- **Порт 5432 занят** — на машине уже стоит PostgreSQL. Поменяйте в compose "5433:5432" и подключайтесь к 5433.
- **Контейнер перезапускается** — `docker compose logs postgres`. Чаще всего — ошибка в SQL из `sql/init`.
- **Данные не загрузились** — init-скрипты выполняются только при первом создании тома. `docker compose down -v` и снова `up -d`.
- **Windows** — убедитесь, что включён WSL 2 backend в Docker Desktop; клонируйте репозиторий внутрь WSL, а не в C:\Users.

Часть 2 · Знакомимся с датасетом (20 мин)

Датасет описывает условную компрессорную станцию: три станции, семь агрегатов, датчики на каждом, сутки телеметрии с шагом в одну минуту, журнал событий и ремонтов. Данные сгенерированы — но структура и типичные «грязные» места в них настоящие.

2.1. Схема

Таблица	Что хранит	Ключевые столбцы
station	Площадка	id, name, region
unit	Агрегат (ГПА, насос...)	id, station_id → station, model, commissioned
sensor	Датчик на агрегате	id, unit_id → unit, kind (temp / vibration / pressure / load), unit_of_measure
telemetry	Измерения, ~40 000 строк	ts, sensor_id → sensor, value
event	События и аварии	id, unit_id → unit, ts, severity (info / warning / alarm / unplanned_stop), message
maintenance	Ремонты и ТО	id, unit_id → unit, performed_at, engineer, parts, notes

2.2. Осмотритесь

Прежде чем писать запросы, потратьте пять минут на то, чтобы понять данные. Ответьте себе на вопросы и запишите ответы в комментариях в начале `sql/seminar01.sql`:

- Сколько строк в каждой таблице? (`select count(*) from ...`)
- Какой период покрывает телеметрия? (`min(ts), max(ts)`)
- Какие значения встречаются в `sensor.kind` и `event.severity`? Есть ли среди них подозрительные — с другим регистром, с пробелом, с опечаткой?
- Есть ли датчики без измерений? Агрегаты без датчиков?
- Найдите хотя бы одну «грязную» запись в `maintenance` — она там точно есть.

Полезные команды psql: \dt — список таблиц, \d telemetry — структура таблицы, \timing on — показывать время выполнения.

Часть 3 · Первые запросы (30 мин)

Решения — в файл sql/seminar01.sql, каждый запрос под комментарием -- Задача N (заготовка уже в файле). Рядом с каждой задачей в заготовке указан ожидаемый результат — сверяйтесь с ним. Задачи 1-7 нужны для чек-поинта.

№	Задача	Что сделать	Приём
1	Агрегаты и площадки	Выведите все агрегаты: unit.id, unit.model, station.name. Отсортируйте по названию площадки, затем по id агрегата.	JOIN, ORDER BY
2	Сколько датчиков на агрегате	Для каждого агрегата — количество датчиков. Агрегаты без датчиков тоже должны попасть в результат (с нулём).	LEFT JOIN, GROUP BY, COUNT
3	Средняя температура за сутки	Для каждого агрегата — средняя температура по всем его датчикам типа temp за весь период. Округлите до одного знака.	JOIN через две таблицы, AVG, ROUND
4	Максимум по каждому датчику	Для каждого датчика — sensor.id, kind, max(value). Выведите только те, у которых максимум превышает 85 (для temp) — остальные типы игнорируйте.	WHERE + HAVING: чем они отличаются?
5	Почасовой профиль	Для агрегата GPA-2 и его датчика температуры подшипника — среднее значение по часам: hour, avg_temp. Отсортировано по часу.	date_trunc, GROUP BY по выражению
6	События по серьёзности	Сколько событий каждого уровня severity произошло на каждой площадке. Формат: station_name, severity, cnt.	GROUP BY по двум столбцам
7	Агрегаты без аварий	Список агрегатов, у которых за период не было ни одного события уровня alarm или unplanned_stop.	NOT EXISTS или LEFT JOIN ... IS NULL — попробуйте оба

Пример оформления

```
-- Задача 1. Агрегаты и площадки
select u.id, u.model, s.name as station
from   unit u
join   station s on s.id = u.station_id
order by s.name, u.id;
```

Прогнать весь файл целиком и посмотреть результаты одной командой:

```
make run          # выполняет sql/seminar01.sql в контейнере и печатает результаты
```

Часть 4 · Репозиторий и команда (15 мин)

4.1. Структура репозитория

```
kddb-seminar01/
├── docker-compose.yml
├── Makefile                # make up / make psql / make run / make reset
├── sql/
│   ├── init/              # схема и датасет – не редактировать
│   │   ├── 01_schema.sql
│   │   └── 02_data.sql
│   └── seminar01.sql      # ← ваши решения
└── docs/
```

```
└─ seminar01.pdf      # это задание
└─ README.md          # ← заполните: кто вы, что заметили в данных
```

4.2. Коммиты и пуш

Делайте небольшие коммиты по мере решения задач — история коммитов будет одним из критериев оценки индивидуального вклада в проекте, поэтому привыкайте сразу. Один коммит на одну-две задачи — нормальный ритм.

```
git add sql/seminar01.sql README.md
git commit -m "seminar01: tasks 1-4"
git push
```

Сдача: покажите репозиторий преподавателю на семинаре (открытый `seminar01.sql` и `make run`) или пришлите ссылку на репозиторий / архив в чат до семинара 2. Автопроверки на этой неделе нет — смотрим руками и обсуждаем.

4.3. Команда и проект

Соберите команду из 2-3 человек и выберите предметную область проекта. Напишите состав команды и область в чат до следующего семинара:

Область	Пример объекта	Что будет в телеметрии
Промышленный объект	насосная станция, компрессорный цех, ГПА	давление, температура, вибрация, нагрузка
Беспилотник / робот	квадрокоптер, мобильный робот, манипулятор	координаты, скорости, токи двигателей, заряд
Умное здание	учебный корпус, ЦОД, склад	температура, CO2, энергопотребление, доступ
Энергетика	подстанция, солнечная станция, ветропарк	напряжения, токи, частота, генерация
Медицинская техника	прикроватный монитор, аппарат ИВЛ	пульс, SpO2, давление, тревоги
Своя	по согласованию с преподавателем	должна быть телеметрия, события и документация

Область можно поменять до недели 3 — дальше схема начнёт обростать деталями, и менять будет дорого.

Часть 5 · Со звёздочкой (если осталось время)

На балл не влияют. Это разминка перед неделей 3 (продвинутый SQL) — оконные функции и работа с «грязными» данными.

№	Задача	Что сделать	Приём
8*	Последнее значение	Для каждого датчика — последнее по времени измерение: <code>sensor_id</code> , <code>ts</code> , <code>value</code> . Одной строкой на датчик.	<code>DISTINCT ON</code> или <code>ROW_NUMBER() OVER</code>
9*	Рост за час	Для датчика температуры подшипника GPA-2 найдите часы, в которых средняя температура выросла более чем на 5 °C относительно предыдущего часа.	<code>LAG() OVER (ORDER BY hour)</code>
10*	Ремонт после аварии	Для каждого события <code>unplanned_stop</code> — был ли ремонт того же агрегата в течение 48 часов после него, и через сколько часов.	<code>JOIN</code> по условию на интервал, <code>EXTRACT(EPOCH ...)</code>

№	Задача	Что сделать	Приём
11 *	Чистим maintenance	Найдите записи о ремонтах, которые, судя по дате, агрегату и тексту, дублируют друг друга. Предложите запрос, который оставляет одну на группу. Не удаляйте — только SELECT.	lower(), trim(), similarity — что считать «одинаковым»?

Что сдаётся и как оценивается

Критерий	Проверяется	Баллы
Стек поднимается	docker compose up -d без ошибок из чистого клона вашего репозитория	0.5
Задачи 1–7	Запросы выполняются без ошибок и дают ожидаемый результат (сверяемся вместе)	1.0
Наблюдения о данных	В начале seminar01.sql или в README — ответы на вопросы части 2.2, найдена хотя бы одна «грязная» запись	0.5
Итого		2.0

Сдача в срок — до начала семинара 2. После — половина баллов. На семинаре 2 я попрошу двух-трёх человек рассказать, как они решали задачу 7 и что нашли «грязного» в данных.

К следующей неделе

- Чек-поинт 1 сдан: показан на семинаре или ссылка / архив прислан в чат.
- Команда собрана, состав и предметная область отправлены в чат.
- Прочитано: Клеппман, «Высоконагруженные приложения», гл. 1–2 (надёжность, масштабируемость; модели данных).
- Подумать: какие сущности будут в *вашей* предметной области. На семинаре 2 будем рисовать ER-диаграмму.

Вопросы по заданию — в чат группы или на консультации. Если что-то в задании непонятно сформулировано — напишите, поправлю и переобновлю файл.